西南交通大學
Southwest Jiaotong University

《智能交通》期末课程设计

设计题目： 大小模型协同的智能交通出行问答系统

成员姓名： 程泽凯　王晨羽　金井直

班　　级： 计算机 2023-04 版

指导教师： 杜圣东

2026 年 6 月 13 日

摘要

本文围绕“大作业 3：大小模型协同的智能交通出行问答系统”要求，设计并实现了一个面向成都市出行服务场景的智能问答原型系统。系统基于 Dify 开发框架进行工作流编排，采用 DeepSeek 开源小参数模型作为基座，并通过 LoRA 微调适配交通问答场景。知识层面，系统采集成都市交管局网站信息发布栏等资料构建本地 RAG 知识库；工具层面，接入高德地图、天气查询和知乎搜索等外部能力；小模型层面，集成大作业 1 STAEformer 交通流预测模型和大作业 2 YOLO 交通目标识别模型，形成覆盖交管信息问答、天气问答、出行建议、路径规划、交通流预测和目标识别的综合应用。前端采用三栏式工作台同步展示回答、证据与可视化结果，后端基于 FastAPI 封装统一接口。测试结果表明，系统能够完成文本问答、路线规划、交通流预测、图片识别和 Dify 工作流调用等核心流程，具备课程设计演示和工程扩展价值。

关键词：智能交通；Dify；检索增强生成；DeepSeek；高德地图；STAEformer；YOLO；交通问答

目录

一、 团队成员分工 1

二、 任务背景 1

三、 任务描述 1

3.1 作业要求对应关系 2

四、 总体技术方案 3

4.1 系统总体架构 3

4.2 关键技术栈 4

4.3 系统功能介绍 5

五、 核心模块设计 5

5.1 意图识别模块 5

5.2 本地 RAG 模块 6

5.3 基座模型选择与模型介绍 8

5.4 大模型微调模块 10

5.5 知乎搜索补充模块 14

5.6 路线规划与地图模块 14

5.7 交通流预测模块 15

5.8 交通标志识别模块 15

5.9 Dify 工作流模块 15

六、 系统实现 19

6.1 前端实现 19

6.2 后端实现 19

6.3 配置与密钥管理 20

6.4 一键启动脚本 21

七、 关键接口设计 22

八、 测试与验证 22

8.1 启动验证 22

8.2 功能测试用例 22

8.3 离线评测结果 23

8.4 端到端延迟测试 25
8.5 界面与运行截图 26
8.6 错误处理验证 30
九、 结果分析 30
十、 不足与改进方向 31
十一、总结 32
参考文献 32

一、 团队成员分工

系统实现与报告整理按以下模块分工：

- 程泽凯：负责需求分析、交通知识库整理、RAG 检索方案和知乎搜索补充策略设计。
- 王晨羽：负责 FastAPI 后端、工具接口、高德 API、交通流预测与 YOLO 识别模块集成。
- 金井直：负责三栏前端界面、右侧证据可视化、Dify 工作流设计、一键启动脚本和报告排版。

二、 任务背景

随着城市交通规模持续扩大，居民出行需求、交通管制信息、路线规划、天气影响和道路风险提示等信息越来越分散。传统交通信息系统通常以公告、地图、热线或单一查询工具形式存在，用户需要在多个平台之间切换，才能获得相对完整的出行决策依据。对于普通驾驶人而言，“成都尾号限行怎么规定”、“雨天开车要注意什么”、“从天府广场到春熙路怎么走”、“某路段未来一小时会不会拥堵” 等问题往往需要同时参考政策、地图、天气和经验性建议。

大语言模型具备自然语言理解和生成能力，但如果单独依赖模型参数回答交通政策和实时出行问题，容易出现事实过期、依据不明和幻觉回答等问题。因此，智能交通问答系统需要将大模型与外部知识库、实时 API、小模型预测和视觉识别模块结合起来，使回答既自然可读，又具备可追溯证据和结构化工具结果。

根据课程设计要求，大作业 3 不仅需要完成普通出行问答，还需要体现 “Dify 开发框架 + DeepSeek 开源小参数基座模型 + 网络采集知识库 + 微调 + 前两次大作业成果集成” 的完整链路。因此，本课程设计在原有大作业基础上进行综合集成：一方面采集成都市交管局网站信息发布栏内容，并结合交通法规、出行建议资料构建本地 RAG 出行知识库；另一方面复用大作业 1 交通流预测成果和大作业 2 交通目标识别成果，并通过 Dify 工作流进行可视化编排。系统最终目标不是单一算法指标优化，而是构建一个能够演示完整智能交通业务闭环的工程原型。

三、 任务描述

本项目任务可以概括为：基于 Dify 开发框架，采用 DeepSeek 开源 1–7B 级模型作为基座模型，构建面向成都市交通出行场景的智能问答系统，支持多意图问题识别、多工具调用、多模态图片输入和可视化证据展示。系统需要满足以下要求：

1. 网络采集成都市交管局网站信息发布栏内容，并结合交通法规、出行建议资料建立本地 RAG 出行知识库。

2. 采用 DeepSeek 开源 1–7B 级模型作为基座模型，并进行面向交通问答场景的 LoRA/QLoRA 微调。
3. 基于 Dify 开发框架构建可视化分支工作流，实现意图识别、工具编排和最终回答生成。
4. 支持交管信息问答、天气问答、出行建议、出行路径规划等基础问答功能。
5. 接入知乎搜索 API，在本地知识库之外提供外部补充信息，并明确标注其参考性质。
6. 接入高德地图 API，实现路线规划结果获取和前端地图路线绘制。
7. 集成大作业 1 STAEformer 交通流预测能力，对成都演示地点进行未来一小时交通流趋势展示，并支持交通流预测问答。
8. 集成大作业 2 YOLO 交通目标/交通标志识别能力，支持用户上传图片并进行识别理解问答。
9. 构建三栏式 HTML 问答界面，使回答、证据和可视化结果同步展示。
10. 提供一键启动和停止脚本，方便课程演示和复现实验。

与传统聊天机器人不同，本系统强调 “回答生成” 与 “证据展示” 分离。大模型负责组织语言，工具和 RAG 负责提供事实依据，前端 panel 负责把结构化结果转化为可视化界面。这样既能提升用户体验，也便于检查回答来源。

3.1 作业要求对应关系

为保证报告内容与课程设计要求一一对应，表1 给出了系统实现对作业要求的覆盖情况。

表 **1**　大作业 **3** 要求与系统实现对应关系

作业要求	本系统实现
基于 Dify 开发框架	构建中文节点分支式 Dify Workflow，通过后端代理调用工作流并复用本地工具接口。
DeepSeek 1–7B 基座模型	选用 `DeepSeek-R1-Distill-Qwen-7B`，并导入为本地 `travelqa` 模型。模型卡标注参数量为 7.62B，工程上按 7B 级蒸馏模型处理；若严格按不超过 7.00B 字面约束，可替换为同系列 1.5B 或 `deepseek-llm-7b-chat`。
网络采集成都市交管局信息发布栏	将成都市交管局信息发布栏、交通法规和出行建议资料清洗切分后写入本地 RAG 知识库。
构建本地 RAG 出行知识库	使用 Hybrid RAG 检索策略，结合语义检索、BM25 和证据片段展示。
对基座模型进行微调	在 AutoDL 环境中完成 LoRA/QLoRA 微调，并通过 SwanLab 记录训练过程。
交管信息、天气、建议、路径规划问答	通过意图识别分别调用 RAG、天气工具、高德路线规划和大模型回答生成。
集成交通流预测模型成果	复用大作业 1 STAEformer，支持成都地点到 PEMS08 节点的演示映射和未来一小时预测。
集成交通目标识别模型成果	复用大作业 2 YOLO 权重，支持图片上传、目标检测、标注图展示和识别理解问答。
完成作业报告	按课程设计模板生成本报告，并提供可复现实验步骤和测试结果。

四、 总体技术方案

4.1 系统总体架构

系统由前端界面、FastAPI 后端、本地 RAG、外部工具、小模型模块和 Dify 工作流组成。整体架构如图1 所示。

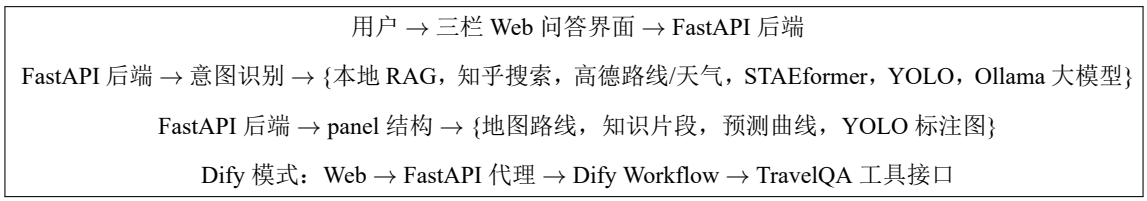


图 **1**　智能交通问答系统总体架构

系统保留本地模式和 Dify 模式两条链路。本地模式由前端直接请求 TravelQA 后端完成问答；Dify 模式由后端代理调用 Dify 工作流，避免在浏览器中暴露应用密钥。无

论采用哪种模式，右侧可视化面板均使用本地 TravelQA 后端生成的 panel 数据，保证用户看到的地图、RAG 片段、预测图和识别结果形式一致。

4.2 关键技术栈

系统主要技术栈见表2。

表 **2**　系统关键技术栈

层级	技术或工具	作用
前端	原生 HTML/CSS/-JavaScript	三栏问答界面、地图、图表和图片展示
后端	FastAPI、Uvicorn	问答接口、工具接口、静态资源托管
大模型	DeepSeek-R1-Distill-Qwen-7B、Ollama 本地模型 travelqa	作为 1–7B 级基座和微调后推理模型，根据证据和工具结果生成自然语言回答
检索	BGE-M3、BM25、Reranker	面向成都市交管局信息发布栏和交通法规资料的本地知识库 Hybrid RAG
外部搜索	知乎搜索 API	本地知识库之外的补充参考
地图服务	高德 Web 服务 API、JS API	路线规划、地理编码和前端地图绘制
预测模型	STAEformer	未来一小时交通流趋势预测
视觉模型	YOLO、TT100K	交通目标/交通标志识别和标注图生成
工作流	Dify 1.14.2	可视化意图分支和工具编排
部署脚本	PowerShell、Docker Compose	一键启动、停止与健康检查

4.3 系统功能介绍

系统围绕成都市出行问答场景，将基础问答、外部工具、小模型预测和多模态识别统一到同一个交互界面中。用户不需要理解底层模型或 API 的调用方式，只需要用自然语言提问，系统会根据意图自动选择处理链路，并在右侧同步展示证据和可视化结果。

表 **3**　系统主要功能说明

功能类别	典型问题或输入	系统处理方式
交管信息问答	成都尾号限行、交通管制、违法处罚等	优先检索本地 RAG 知识库，必要时补充知乎搜索，并提示以官方发布为准
天气问答	成都今天是否下雨、雨天出行是否安全	调用天气工具获取天气信息，再由大模型生成出行提示
出行建议	下雨天开车要注意什么、事故后如何处理	检索法规和安全驾驶建议，结合大模型生成结构化建议
出行路径规划	从天府广场到春熙路怎么走	抽取起终点，调用高德路线规划 API，右侧展示地图路线和步骤
交通流预测问答	春熙路未来一小时交通流如何	将成都地点名映射到 PEMS08 节点，调用 STAEformer 预测并绘制折线图
交通目标识别理解	上传交通标志图片	调用 YOLO 识别交通标志，生成标注图、类别、置信度和解释回答
Dify 工作流模式	切换到 Dify 后继续提问	通过 Dify 分支工作流完成意图识别和工具编排，前端仍展示本地证据面板

五、 核心模块设计

5.1 意图识别模块

意图识别模块通过规则关键词将用户问题映射到不同处理路径。当前支持的主要意图包括：

- `traffic-info`：交通政策、限行、违法处罚、管制等信息。
- `travel-advice`：雨天驾驶、事故处理、出行建议等。
- `weather`：天气、气温、降雨、能见度等。
- `route`：从某地到某地的路线规划和导航。
- `forecast`：交通流、车流、拥堵趋势和未来预测。
- `detection`：交通标志识别、YOLO、图片检测能力。

规则路由的优点是可解释、稳定且便于课程演示。对于路线规划问题，系统还会调用 `extract_route_od` 从自然语言中抽取起点和终点。例如用户输入“我从天府广场到

春熙路怎么走”时，系统会解析出起点为天府广场、终点为春熙路，并进入路线规划工具。需要说明的是，本地 FastAPI 原型采用关键词规则，是为了保证离线复现和便于调试；Dify 工作流中的“意图识别”节点可以进一步替换为 LLM 分类节点，要求模型只输出 `route`、`forecast`、`weather`、`detection`、`traffic-info` 或 `travel-advice` 六类标签。这样既保留本地演示的确定性，也为后续使用微调模型做意图分类留出接口。

本地路由核心代码如下。规则顺序采用“强工具信号优先”：路线、预测、识别、天气先匹配，限行和违章等交通政策关键词再进入 RAG，最后兜底到 `traffic-info`。修订过程中还补充了“从 A 到 B 堵不堵”、“实时路况”、“避堵路线”等路线规划信号，避免将实时路线问题误分到普通 RAG。

```
_RULES = [
    (Intent.ROUTE, ["怎么去", "怎么走", "路线", "路径", "导航",
                    "从.*到.*堵", "实时路况", "避堵路线"]),
    (Intent.FORECAST, ["交通流", "预测", "未来.*车流", "mae", "staeformer"]),
    (Intent.DETECTION, ["识别", "检测", "标志", "yolo", "tt100k"]),
    (Intent.WEATHER, ["天气", "气温", "降水", "会不会下雨"]),
    (Intent.TRAFFIC_INFO, ["限行", "尾号", "违章", "管制", "施工"]),
    (Intent.TRAVEL_ADVICE, ["建议", "注意", "事故", "快处", "追尾"]),
]

def classify(query: str) -> Intent:
    q = query.lower()
    for intent, patterns in _RULES:
        for pat in patterns:
            if re.search(pat, q):
                return intent
    return Intent.TRAFFIC_INFO
```

5.2 本地 RAG 模块

RAG 模块负责把本地交通知识库中的交管公告、法规条文和出行建议检索出来，为大模型回答提供依据。按照作业要求，知识库建设重点面向成都市交通出行场景，数据来源包括网络采集的成都市交管局网站信息发布栏内容、道路交通安全相关法规、常见驾驶安全建议和项目整理的出行问答材料。对政策公告类信息，系统在 chunk 元数据中保留标题、来源信息、发布时间或抓取时间，便于前端右侧证据面板展示和用户核验。

系统采用 Hybrid Search 策略，综合语义向量检索和 BM25 关键词检索。其设计动机是交通问答中包含大量具体路名、政策关键词和日期信息，单纯语义检索容易出现语义相近但事实不匹配的问题。

本地 RAG 流程如下：

1. 网络采集成都市交管局信息发布栏，并导入交通法规、公告网页、出行建议或 PDF。

2. 文档清洗与格式统一，将网页正文、标题、发布时间、来源信息转换为结构化文本。
3. 文本切分为 chunk，保留标题、发布时间、来源和正文片段。
4. 使用 BGE-M3 或离线哈希向量生成文本向量。
5. 构建 BM25 关键词索引。
6. 查询时同时执行向量检索和关键词检索。
7. 使用 RRF 或重排序模型融合结果。
8. 将命中片段作为 evidence 输入大模型。

从工程实现角度看，RAG 不只是简单的文本搜索，而是一个从数据采集到回答生成的闭环流程。其完整链路见表4。

表 **4**　本地 **RAG** 全流程

阶段	输入/输出	说明
数据采集	成都市交管局信息发布栏、法规网页、PDF、出行建议资料	采集时保留来源信息、标题、发布时间或抓取时间，便于后续追溯
文档清洗	原始 HTML/PDF → 纯文本	去除导航栏、页脚和重复文本，统一编码和段落结构
结构化切分	文档 → chunks	按语义段落切分，并保留 category、district、road、valid_time 等元数据
索引构建	chunks → 向量索引与 BM25 索引	语义检索负责相近表达匹配，BM25 负责政策关键词、路名和时间等精确匹配
查询理解	用户问题 → 意图和检索 query	识别交通政策、出行建议、天气、路线、预测或识别等意图
混合检索	query → 候选片段	同时执行向量检索和关键词检索，避免单一路径漏召回
结果融合	候选片段 → Top-K 证据	使用融合排序或重排序策略筛选最相关片段
外部补充	query → 知乎搜索片段	即使本地知识库命中，也补充搜索一次，但在回答中明确其仅供参考
回答生成	evidence + query → answer	大模型基于本地证据优先生成回答，不把外部补充当作官方结论
证据展示	chunks → panel	前端右侧展示标题、摘要、来源和时间信息，方便用户核验

前端右侧会展示本轮命中的知识片段，包括标题、正文摘要、来源信息、发布时间或抓取时间。这样用户不仅能看到回答，还能核验回答依据。

当前知识库与检索配置见表5。知识库索引文件 `data/kb/index.json` 中保留每个 chunk 的 `id`、`text`、`title`、`url`、`fetched_at`、`published_at`、`category`、`district`、`road` 和 `valid_time` 等元数据，因此前端能够展示出处而不只是展示模型回答。

表 5　RAG 知识库与检索配置

项目	当前配置或结果
原始来源文档	18 篇，包括成都市交管局公告索引、交通法规、开放交通数据和出行建议资料
知识片段数	287 个 chunk
向量模型	`models/rag/bge-m3`
Embedding provider	`sentence_transformers`
检索模式	`hybrid`，向量检索 + BM25，RRF 融合
最终 Top-K	`top_k=4`，重排序后送入大模型
重排序模型	`models/rag/bge-reranker-v2-m3`
过滤阈值	`min_score=0.05`

Hybrid Search 的核心实现如下。系统先分别得到向量召回和 BM25 召回，再使用 Reciprocal Rank Fusion 将两路排名融合，最后把候选交给 Reranker 做二次排序。

```
def hybrid_search(self, query, query_vec, top_k=4,
                  vector_top_k=20, bm25_top_k=20, rrf_k=60):
    vec_hits = self.vector_search(query_vec, top_k=vector_top_k)
    bm25_hits = self.bm25_search(query, top_k=bm25_top_k)
    fused = {}
    for rank, (idx, _) in enumerate(vec_hits):
        fused[idx] = fused.get(idx, 0.0) + 1.0 / (rrf_k + rank + 1)
    for rank, (idx, _) in enumerate(bm25_hits):
        fused[idx] = fused.get(idx, 0.0) + 1.0 / (rrf_k + rank + 1)
    ranked = sorted(fused.items(), key=lambda x: x[1], reverse=True)
    return [self._to_hit(sc, self._meta[idx]) for idx, sc in ranked[:top_k]]
```

5.3 基座模型选择与模型介绍

本项目的大模型基座选用 `DeepSeek-R1-Distill-Qwen-7B`。该模型来自 DeepSeek-R1 系列蒸馏模型，可通过 ModelScope 开源模型页面获取。DeepSeek-R1-Zero 首先验证了在不依赖大规模监督微调冷启动数据的情况下，通过强化学习可以激发模型的推理能力，但也存在重复、可读性不足和中英文混杂等问题。DeepSeek-R1 在此基础上引入冷启动数据并继续强化学习，使模型在数学、代码和复杂推理任务上表现更稳定。蒸馏版本则把 DeepSeek-R1 的推理模式迁移到更小规模的稠密模型上，使其更适合本课程设计中的本地化部署和 LoRA/QLoRA 微调。

表 **6** **DeepSeek-R1-Distill-Qwen-7B** 模型卡片

项目	内容
模型名称	DeepSeek-R1-Distill-Qwen-7B
模型来源	ModelScope 开源模型仓库
参数量	7.62B
基础模型系列	Qwen 系列蒸馏基座
权重格式	Safetensors
深度学习框架	PyTorch
张量类型	BF16
开源协议	MIT
模型大小	15.24 GB
更新时间	2025 年 2 月 25 日

与完整 DeepSeek-R1 相比，7B 级蒸馏模型的参数规模更适合课程设计场景：一方面，它保留了较好的中文理解、步骤化分析和工具结果解释能力；另一方面，模型体积仍处于个人工作站或云端单卡环境可处理范围内，便于进行 QLoRA 训练、Ollama 本地推理和课堂演示。对于本系统而言，大模型并不直接承担实时政策查询或路线计算，而是承担意图理解后的回答组织、RAG 证据综合、工具输出解释和风险提示生成，因此 `DeepSeek-R1-Distill-Qwen-7B` 的推理与表达能力能够较好匹配任务需求。

公开模型介绍中给出的蒸馏模型评测结果如表7 所示。可以看到，该模型在 MATH-500 和 AIME 等推理任务上具有较强表现，说明其具备一定复杂问题拆解能力。虽然交通问答并不等同于数学竞赛任务，但路线规划解释、交通流预测趋势说明和法规依据组织同样需要模型对结构化信息进行归纳和稳健表述，因此该能力具有迁移价值。

表 7　**DeepSeek-R1-Distill-Qwen-7B** 公开评测指标

评测项目	指标值
AIME 2024 pass@1	55.5
AIME 2024 cons@64	83.3
MATH-500 pass@1	92.8
GPQA Diamond pass@1	49.1
LiveCodeBench pass@1	37.6
CodeForces rating	1189

官方使用建议中提到，推理时温度可设置在 0.5 至 0.7 区间，并建议减少复杂系统提示词对模型行为的干扰。在本项目中，实际推理链路采用“证据与工具结果先行、模型负责组织语言”的方式：本地 RAG、知乎搜索、高德 API、STAEformer 和 YOLO 先产生结构化 evidence，再由本地 `travelqa` 模型生成最终回答。为了避免前端出现冗长推理过程，Ollama provider 中固定使用 `think=false`，同时在提示词中约束模型只输出面向用户的结论、依据和注意事项。

5.4 大模型微调模块

除本地 RAG 与工具调用外，项目还完成了面向交通问答场景的大模型微调环节。微调基座模型为 `DeepSeek-R1-Distill-Qwen-7B`，训练方式采用 LoRA/QLoRA 思路，即冻结大部分基座参数，仅训练低秩适配器参数，从而在有限显存和较短训练时间内完成领域适配。微调目标不是让模型死记交通公告，而是让模型学习交通问答的表达格式、工具结果解释方式、RAG 证据引用方式以及无法确认时的稳健回答策略。

训练数据由离线脚本生成，统一保存为 Alpaca 风格 JSONL。每条样本至少包含 `instruction`、`input` 和 `output` 三个字段，并附加 `category`、`source` 和 `group_id` 用于审计。数据构造过程中使用 `group_id` 保证同一事实或同一工具场景的释义变体不会同时出现在训练集和验证集中，从而避免语义泄漏。

```
{
  "instruction": "成都公共数据开放平台是否收录某交通资源？怎样获取？",
  "input": "",
  "output": "开放目录元数据显示，该资源由对应部门提供。目录元数据不等于数据正文。",
  "category": "open_catalog",
  "source": "chengdu-open-catalog",
```

```
  "group_id": "open_catalog:..."
}
```

微调数据集统计见表8。可以看到，数据并非只包含少量手写问答，而是覆盖公交、轨道、停车、开放目录、路线工具、天气工具、预测、识别、拒答和工具不可用等多类边界场景。

表 **8**　**LoRA/QLoRA** 微调数据集统计

项目	数值或说明
样本总数	1228 条 SFT 样本
训练集 / 验证集	1104 / 124
数据划分随机种子	20260601
group 泄漏检查	0，训练集和验证集无同组语义泄漏
主要类别	bus 320、metro 252、parking 198、open_catalog 160、refusal 80、tool_call 40、toll_station 38
高阶集成类别	forecast 20、detection 20、route 8、weather 8、tool_unavailable 8
数据源审计	新都公交 87 行；成都市开放目录原始 7408 行，筛选 160 行进入 SFT

本次训练在 AutoDL 容器环境中完成，训练脚本加载 DeepSeek 基座模型和交通问答训练集，按 LoRA/QLoRA 参数进行两轮训练，并在训练结束后保存领域适配器。

训练过程使用 SwanLab 进行实验跟踪。SwanLab 记录的关键信息如下：

- 实验名称：DeepSeek 交通问答 LoRA 微调实验。
- 运行标识：SwanLab 自动生成的本次实验记录编号。
- 开始时间：2026 年 6 月 2 日 10:21:50
- 运行时间：5 分钟 19 秒
- SwanLab 版本：0.7.19
- 输出结果：训练日志、评估指标和 LoRA 适配器。

训练环境为 Ubuntu 22.04.5 LTS，Python 版本为 3.12.3。训练共进行 2 个 epoch，总步数为 138 步。训练日志显示，loss 从初始阶段的 3.6966 快速下降，在第 1 个 epoch 附近稳定到约 1.0 左右，说明模型较快学习到交通问答数据的基本格式与输出模式。训练最终指标见表9。

表 9　**DeepSeek-R1-Distill-Qwen-7B LoRA** 微调指标

指标	数值
训练轮数	2 epoch
总训练步数	138 steps
训练运行时间	313.7957 s
train_samples_per_second	7.036
train_steps_per_second	0.44
train_loss	1.1065
第 1 轮 eval_loss	0.9677
第 2 轮 eval_loss	0.9193
eval_samples_per_second	16.055
eval_steps_per_second	16.055

从训练曲线看，loss 在前半个 epoch 内下降明显：0.07 epoch 时 loss 为 3.6966，0.36 epoch 时降至 1.1901，0.72 epoch 时进一步降至 0.9854。第 2 个 epoch 中，loss 多数维持在 0.6 至 0.9 区间，最终训练平均损失为 1.1065。验证损失从第 1 轮的 0.9677 降至第 2 轮的 0.9193，说明微调在验证集上仍有正向收益，未出现明显发散。梯度范数整体处于约 0.79 至 1.52 之间，训练过程较稳定。

微调后的 LoRA 适配器可以与基座模型合并或通过推理时加载适配器的方式使用。在本地项目中，最终通过 Ollama 导入为 `travelqa` 模型，并由 `config.yaml` 中的 `ollama_native` provider 调用。这样，系统的回答生成不再完全依赖通用基座模型，而是具备面向成都交通问答、工具结果解释和 RAG 证据组织的领域表达能力。

为了验证微调是否只是在训练集上降低 loss，项目还提供 `finetune/ab_eval.py`，构建 100 条 holdout 问题，对未微调基座模型和加载 LoRA 后模型分别生成回答，再导出 CSV 与 HTML 页面供人工评分。当前已生成的 `ab_eval_v4/ab_results.json` 中，100 条问题的基座和微调后输出均已保存。表10 给出部分代表性样例。可以看到，基座模型容易直接编造实时路线、天气或公告细节；微调后模型更倾向于要求先调用工具、引用公告来源或在缺少图片时拒绝臆断。

表 **10**　微调前后代表性输出对比

问题	未微调基座模型倾向	加载 LoRA 后倾向
现在从成都东站开车到春熙路大约需要多久？	直接给出约 20–30 分钟，并展开路线猜测。	要求先调用 `route_plan`，收到工具结果后再回答，不编造距离或耗时。
成都今天下午会下雨吗？出门需要带伞吗？	编造温度、风向和降雨概率。	说明不能凭空判断未来天气，需要查询实时天气或气象台预报。
请介绍最近的成都交管施工绕行通知，并给出可核验来源。	输出占位式的绕行路线、实施时间和政策范围。	要求先检索公告或调用工具，不能编造道路、限行或收费信息。
没有上传图片时，能判断前方一定有禁止停车标志吗？	能给出拒答，但仍带有较长推理过程。	明确说明缺少图片无法确认，应补充图片并结合实时路况核验。

在样例对比之外，本文进一步使用 `eval/score_ab_eval.py` 对 100 条 holdout 输出进行规则化评分。评分规则只检查本次微调明确优化的行为：是否显式要求调用工具、查询实时来源或拒绝不当请求，是否出现疑似编造的实时数值，是否泄漏 `<think>` 推理过程。该评分不是人工语义裁判，但可复现、可审计，能量化反映微调是否改善了工具调用意识和回答边界。结果见表11。

表 **11**　微调前后 **100** 条 **holdout** 规则化评分

指标	未微调基座	加载 LoRA 后
样本数	100	100
合规回答数	22	90
合规率	22.0%	90.0%
疑似编造数值条数	20	0
疑似编造率	20.0%	0.0%
`<think>` 泄漏条数	60	0

从类别看，LoRA 在路线工具、天气工具、停车实时余位、隐私拒答、工具不可用、公告依据和图片识别等场景提升最明显。例如路线工具类 12 条中，微调后 12 条均要求先调用路线工具或避免编造耗时；隐私拒答类 10 条中，微调后 10 条均明确拒绝提供个人车主、轨迹或联系方式信息。少数开放目录和预测类问题仍有提升空间，主要原因是规则化评分要求回答中显式出现工具或核验词，而部分正确回答可能使用了更自然的描述。

为弥补规则化评分不能判断整体回答质量的问题，本文从 100 条 holdout 输出中抽取 20 条覆盖路线、天气、停车、隐私、工具不可用、公告依据、公交地铁、预测和图片识别的小样本，按 helpfulness、依据/工具意识、边界控制三个维度进行人工盲评复核。评分文件为 `eval/ab_blind_review_20.json`，摘要脚本为 `eval/summarize_ab_blind_review.py`。结果见表12。

表 **12**　微调前后 **20** 条人工盲评复核

指标	未微调基座	加载 LoRA 后
平均质量分（0–5）	1.65	4.25
胜出样本数	1	17
平局样本数	2	
主要问题	编造实时数值、泄漏推理	个别回答过短或工具名不够精确

人工复核结果与规则化评分方向一致：微调后模型在不编造实时结果、拒绝隐私请求和要求调用工具方面明显更稳。但也暴露了两个边界样例：一是个别停车问题中工具建议不够贴切，二是交通标志置信度解释问题上，微调后回答过于保守，缺少对置信度含义的解释。这些样例已在评审 JSON 中保留，便于后续继续优化数据集。

5.5 知乎搜索补充模块

系统使用知乎开放 API 进行站内搜索，搜索结果包括标题、摘要、来源地址、作者、赞同数、评论数和更新时间。该模块的设计目标不是替代本地知识库，而是在本地知识库不足或用户问题偏经验性时提供补充参考。

系统规定本地知识库优先级高于知乎搜索。即使本地知识库已有命中，系统也会补充进行一次知乎搜索，但在回答中明确标注：

- 知乎搜索内容仅供参考。
- 不代表官方结论。
- 对时效性政策、处罚和临时管制，以官方发布为准。

对于“成都尾号限行怎么规定”一类强时效问题，系统会检查知乎结果时间。较旧结果会被归入历史参考，不作为当前规则直接输出，避免把过期内容误认为现行政策。

5.6 路线规划与地图模块

当系统识别到路线意图后，后端会调用高德 Web 服务 API 完成 POI 解析、地理编码和驾车路径规划。路线结果包括起点、终点、距离、预计耗时、导航步骤和 polyline 坐标串。

前端只获取地图绘制所需的安全配置，并在右侧面板中使用高德地图路线规划能力绘制导航线路。如果前端地图配置缺失，页面不会崩溃，而是使用后端返回的路线坐标生成降级简图，并继续展示路线摘要。

路线规划模块体现了大模型与确定性工具的分工：路径距离、耗时和路线步骤由高德 API 提供，大模型只负责解释结果和组织回答。

5.7 交通流预测模块

交通流预测模块复用大作业 1 中的 STAEformer 结果。由于当前项目面向成都演示场景，而底层预测数据来自 PEMS08，因此系统设计了固定的 “成都地点名到 PEMS08 node_id” 映射。用户可以使用熟悉的成都地名进行提问，系统再映射到底层节点完成预测。

例如：

- 春熙路映射到 node_id=12。
- 成都东站映射到 node_id=24。
- 熊猫基地映射到 node_id=108。

系统会在回答和右侧面板中明确说明该映射是演示用模拟映射，不是真实成都传感器数据。右侧面板会绘制未来 60 分钟交通流折线图，并展示趋势信息，如 rising、falling 或 stable。

5.8 交通标志识别模块

交通标志识别模块复用大作业 2 的 YOLO 交通标志检测模型。用户上传图片后，系统先完成图片接收与临时存储，再调用检测模型进行推理，并生成带框标注图供右侧面板展示。

检测结果包括：

- 交通标志类别。
- 置信度。
- 边界框坐标。
- 标注图访问地址。

前端右侧显示标注图和检测列表。若未检测到已训练类别，系统会返回 “未识别到已训练交通标志类别”，而不是输出不确定猜测。

5.9 Dify 工作流模块

Dify 工作流已导出为可在平台中直接导入的工作流格式，便于在课堂验收或新环境中复现实验。工作流共有 16 个节点、19 条边，节点均使用中文名，包括开始、意图识别、按意图分流、路线规划接口、交通流预测接口、本地 RAG 与知乎补充、合并分支回答和最终回复等。Dify 运行在容器中，通过容器到宿主机的内部访问通道调用 TravelQA 后端工具能力。

该工作流不是简单的 HTTP 包装，而是按照问题类型进行分支处理。路线问题调用路线接口，预测问题调用预测接口，天气问题调用天气接口，识别问题调用检测摘要接口，其余问题进入本地 RAG 与知乎补充分支。

Dify 节点职责见表13。这些节点共同完成从用户输入、意图分类、分支调用、结果格式化到最终回复的完整编排过程。

表 **13** **Dify** 工作流节点职责说明

节点名称	节点类型	作用说明
开始	Start	接收用户问题、会话信息和前端传入的上下文变量
意图识别	LLM/分类	将自然语言问题分类为路线、预测、天气、识别或 RAG 问答
按意图分流	条件分支	根据意图把请求路由到对应工具节点
路线规划接口	HTTP	调用 TravelQA 后端路线工具，返回起终点、距离、耗时和路径步骤
整理路线回答	Code/LLM	将路线工具 JSON 转换成可读回答和简要摘要
预测参数映射	Code	把成都地点名映射为 PEMS08 node_id 和演示路段名称
交通流预测接口	HTTP	调用 STAEformer 预测接口，获得未来 60 分钟交通流序列
整理预测回答	Code/LLM	解释预测趋势，并说明成都地点名为演示映射
天气接口	HTTP	调用天气工具获取城市天气、温度和降雨等信息
整理天气建议	Code/LLM	将天气结果转换为出行建议
标志识别能力接口	HTTP	获取 YOLO/TT100K 支持类别和训练指标摘要
整理识别说明	Code/LLM	回答系统能识别哪些交通标志及其含义
本地 RAG 与知乎补充	HTTP	调用本地 RAG 与知乎补充链路，返回证据片段和参考来源
整理 RAG 回答	Code/LLM	按本地知识库优先原则组织政策、法规或出行建议回答
合并分支回答	Variable/Code	统一不同分支的回答字段，保证最终节点输入一致
最终回复	Answer	向用户返回本轮最终回答

Dify 的分支执行流程见表14。每条分支都不是孤立回答，而是先获得结构化工具结果，再交给整理节点转为自然语言，最后由统一回复节点输出。前端在 Dify 模式下仍会复用本地后端生成的证据面板，因此用户看到的右侧地图、RAG 片段、预测曲线和

识别结果与本地模式保持一致。

表 **14**　**Dify** 工作流分支流程说明

流程类型	节点流转	输出内容
路线规划流程	开始、意图识别、按意图分流、路线规划接口、整理路线回答、合并分支回答、最终回复	起终点、距离、预计耗时、主要导航步骤和右侧地图路线证据
交通流预测流程	开始、意图识别、按意图分流、预测参数映射、交通流预测接口、整理预测回答、合并分支回答、最终回复	成都地点名、演示映射节点、未来 60 分钟预测序列、趋势判断和折线图
天气出行流程	开始、意图识别、按意图分流、天气接口、整理天气建议、合并分支回答、最终回复	天气状态、温度、降雨信息和面向出行的安全建议
标志识别流程	开始、意图识别、按意图分流、标志识别能力接口、整理识别说明、合并分支回答、最终回复	可识别交通标志类别、检测能力说明和图片识别入口提示
RAG 问答流程	开始、意图识别、普通交通问答分支、本地 RAG 与知乎补充、整理 RAG 回答、合并分支回答、最终回复	本地知识库依据、知乎补充参考、来源优先级说明和政策时效提醒
统一回复流程	各工具分支、合并分支回答、最终回复	将不同分支的结果整理为统一回答字段，保证前端显示稳定

图2 展示了 Dify 工作流的全景结构。可以看到，工作流从开始节点进入意图识别，再通过条件分支路由到路线、预测、天气、识别和 RAG 等不同工具链路，最后汇总到统一回复节点。



图 **2**　**Dify** 工作流全景

图3 展示了路线、交通流预测、天气和标志识别等工具分支。每个工具节点均调用 TravelQA 后端的对应能力，使 Dify 工作流能够复用本地系统已有的小模型和外部 API。

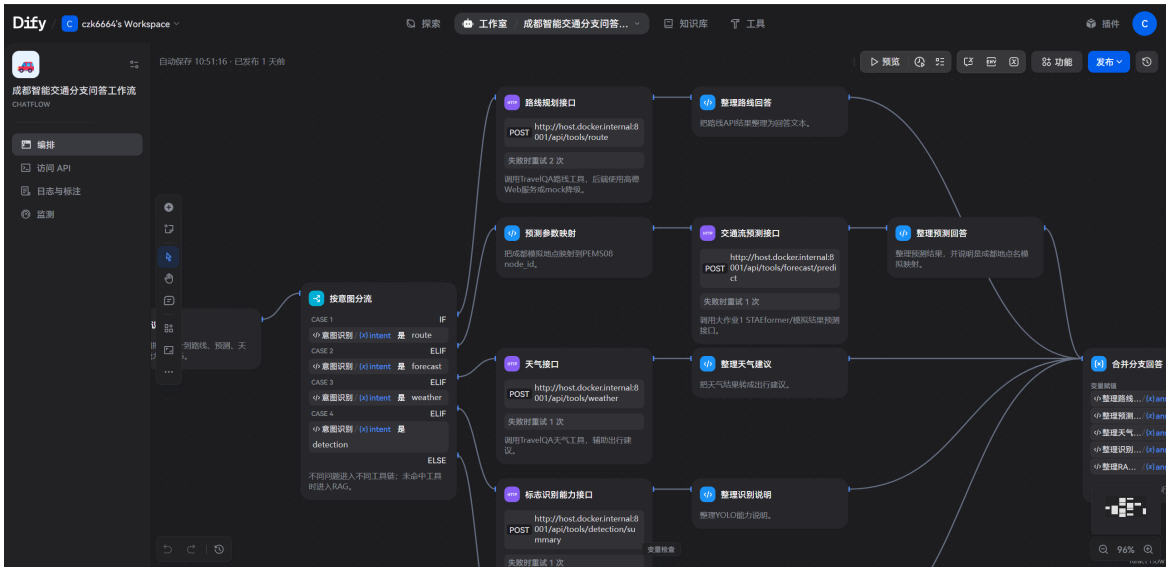


图 **3**　**Dify** 工具调用分支

图4 展示了天气、标志识别和本地 RAG 与知乎补充分支。对于未命中路线、预测、天气或识别的普通交通问答，工作流会进入 RAG 分支，调用本地问答接口获得知识库证据和补充搜索结果。

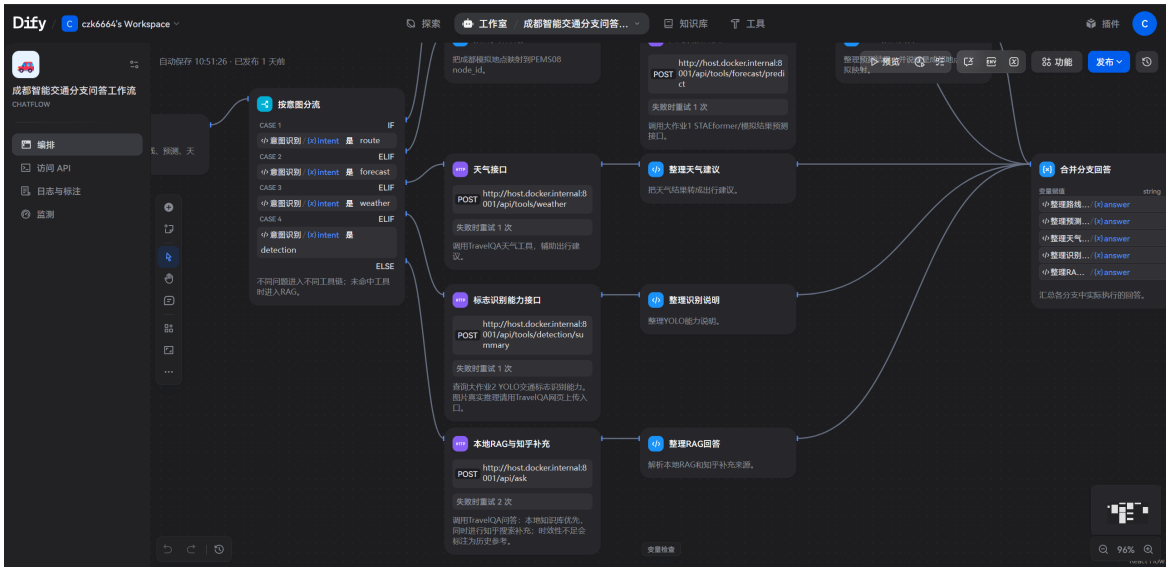


图 **4**　**Dify RAG** 与识别分支

六、 系统实现

6.1 前端实现

为了减少部署复杂度，前端未使用 React/Vue 等框架，而是采用原生 HTML、CSS 与 JavaScript 实现单页应用。页面结构分为三栏：

- 左侧控制台：运行状态、知识库状态、大模型状态、高德地图状态、Dify 状态、回答引擎切换、示例问题和图片上传。
- 中间问答区：展示用户输入和系统回答，回答带有意图标签。
- 右侧证据区：根据 panel 类型展示地图、知识片段、预测图或识别图。

前端通过统一的面板渲染函数处理结构化结果。其核心思想是：后端返回什么类型的 panel，前端就渲染对应的证据形态。这样后端可以不断扩展新工具，而前端只需增加新的 panel 渲染器。

6.2 后端实现

后端基于 FastAPI，核心编排层负责完成意图识别、RAG 检索、工具调用、大模型生成和 panel 构造。系统将普通问答、地图路线、天气查询、交通流预测、图片识别和 Dify 代理统一封装到同一服务中，便于前端和工作流复用。

核心返回结构如下：

```
{
  "intent": "route",
  "text": "回答文本",
  "sources": [],
  "tool_result": {},
  "panel": {}
}
```

`text` 用于中间问答区，`panel` 用于右侧证据与可视化区，`sources` 用于来源摘要，`tool_result` 保留底层工具原始结构，方便调试和后续扩展。

后端核心编排逻辑如下。不同意图对应不同 handler，工具结果和 evidence 先生成，再由大模型或降级模板组织面向用户的回答。

```
intent = classify(query)
handler = {
    Intent.TRAFFIC_INFO: self._rag,
    Intent.TRAVEL_ADVICE: self._rag,
    Intent.WEATHER: self._weather,
    Intent.ROUTE: self._route,
    Intent.FORECAST: self._forecast,
    Intent.DETECTION: self._detection,
```

```
}[intent]
return handler(query, intent)
```

以交通流预测为例，后端会先把成都地名映射到 PEMS08 演示节点，再把预测工具结果写入 `tool_result` 和右侧 `forecast` panel。回答文本中同步加入 `simulation_note`，明确说明该结果是演示映射而非真实成都传感器预测。

```
location, node_id = self._forecast_location_node(query)
fc = self.forecast.predict(node_id=node_id)
fc["location"] = location
fc["simulation_note"] = "成都地点名为演示用模拟映射，不是真实成都传感器数据。"
return Answer(
    intent=intent.value,
    text=text,
    evidence=f"交通流预测模型结果：{fc}",
    tool_result=fc,
    panel={"type": "forecast", "forecast": fc},
)
```

Dify 代理接口的关键点是：浏览器不直接持有 Dify App Key，而是由后端读取环境变量并调用 Dify；Dify 负责最终措辞，本地后端再为同一问题补建右侧证据面板。

```
resp = requests.post(
    api_base.rstrip("/") + "/chat-messages",
    headers={"Authorization": f"Bearer {api_key}"},
    json={"query": req.query, "response_mode": "blocking", "user": user},
    timeout=timeout,
)
data = resp.json()
local_context = _local_context_for_panel(req.query)
return {
    "text": data.get("answer") or "",
    "sources": local_context.get("sources") or _dify_sources(data),
    "tool_result": local_context.get("tool_result") or data,
    "panel": local_context.get("panel"),
}
```

6.3 配置与密钥管理

系统将普通配置与敏感密钥分离管理。普通配置用于描述工具开关、模型路径、知识库位置和成都地点映射；地图服务密钥、知乎搜索密钥和 Dify 应用密钥等敏感信息只在本地环境中读取，不写入报告正文，也不直接暴露给浏览器。

为了保证安全，后端不会把高德 Web 服务密钥、知乎搜索密钥或 Dify 应用密钥下发到浏览器。前端只获取绘制地图所需的最小配置，其他外部服务调用均由后端代理完

成。

当前 config.yaml 的核心结构如下。可以看到，模型、RAG、外部工具和前两次大作业集成均通过配置文件声明；敏感 Key 只给出环境变量名，由本机 .env.local 或系统环境变量注入。

```
llm:
  provider: ollama_native
  base_url: http://127.0.0.1:11434
  model: travelqa
embedding:
  provider: sentence_transformers
  model: models/rag/bge-m3
retrieval:
  mode: hybrid
  top_k: 4
  rerank_top_k: 4
  reranker:
    provider: sentence_transformers
    model: models/rag/bge-reranker-v2-m3
tools:
  weather: {provider: amap, api_key_env: TRAVELQA_AMAP_KEY}
  route: {provider: amap, api_key_env: TRAVELQA_AMAP_KEY}
  zhihu_search: {enabled: true, secret_env: TRAVELQA_ZHIHU_API_SECRET}
integrations:
  forecast: {enabled: true, results_dir: 大作业1/results}
  detection: {enabled: true, model_path: 大作业2/.../best.pt}
```

6.4 一键启动脚本

项目提供一键启动和一键停止脚本。启动脚本会自动完成：

1. 检查 Docker 是否可用，必要时启动 Docker Desktop。
2. 启动 Dify Docker Compose 服务。
3. 为 Dify 分配本地访问端口。
4. 启动 TravelQA FastAPI 后端。
5. 检查 TravelQA 和 Dify 是否正常响应。
6. 在 Dify 容器内验证其能访问宿主机上的 TravelQA 后端。

停止脚本会关闭 TravelQA 后端进程和 Dify Compose 容器，但不会删除知识库、模型、工作流配置和数据库数据，便于后续继续演示。

七、 关键接口设计

系统接口见表15。

表 **15**　系统主要服务接口

接口类别	方法	功能
Web 首页	GET	返回三栏智能交通问答界面
健康检查	GET	检查后端、知识库、大模型和工具配置状态
前端配置	GET	返回地图绘制所需的安全前端配置
本地问答	POST	文本问答入口，负责意图识别、RAG、工具调用和回答生成
图片问答	POST	图片上传、YOLO 推理、标注图生成和大模型分析入口
Dify 状态	GET	检查 Dify 应用密钥和工作流服务是否可用
Dify 对话	POST	后端代理调用 Dify 工作流，并保持右侧证据面板一致
天气工具	POST	查询天气并生成出行建议
路线工具	POST	调用地图服务完成地点解析和路线规划
预测工具	POST	调用交通流预测模型获得未来一小时预测序列
识别工具	POST	返回交通标志识别能力说明，供工作流问答使用
静态产物	GET	展示上传图片、检测标注图和可视化结果

八、 测试与验证

8.1 启动验证

系统启动后需要同时验证 TravelQA 本地页面和 Dify 控制台。启动脚本完成后，会给出两个可访问入口，并检查后端健康状态、Dify 页面状态和容器到宿主机的工具调用链路。

脚本验证通过后，说明：

- TravelQA 三栏问答界面可以正常打开。
- Dify 控制台可以正常访问并运行导入的工作流。
- Dify 工作流能够调用本地 TravelQA 后端工具接口。

验证结束后，可使用停止脚本关闭后端服务和 Dify 容器，保留本地数据与配置。

8.2 功能测试用例

测试用例见表16。

表 **16**　核心功能测试用例

用户输入	预期意图	预期结果
我从天府广场到春熙路怎么走？	路线规划	中间返回路线回答，右侧显示地图路线、距离、耗时和导航步骤
下雨天开车要注意什么？	出行建议/RAG	本地知识库优先回答，同时展示知乎补充片段
成都尾号限行怎么规定？	交通信息/RAG	展示本地依据；知乎旧结果作为历史参考，不作为当前规则
我想知道春熙路未来一小时交通流	交通流预测	春熙路映射到 node_id=12，右侧显示预测折线图
上传交通标志图片	标志识别	调用 YOLO，右侧显示标注图、类别和置信度
切换 Dify 后提问路线问题	Dify 工作流	Dify 生成回答，右侧仍显示本地路线面板

8.3 离线评测结果

除界面截图外，项目还提供 `eval/run_eval.py` 作为可复现的离线功能评测脚本。评测集 `eval/eval_set.jsonl` 共 12 条问题，覆盖交通信息、出行建议、天气、路线、交通流预测和交通标志识别六类意图。脚本先检查 `classify(query)` 的预测意图是否正确，再调用 `TravelQAApp.ask(query)`，把回答文本和 evidence 拼接后检查人工指定的关键词是否命中。当前环境重新运行结果见表17。

表 **17**　离线评测结果

指标	结果	说明
意图路由准确率	12/12 = 100.0%	六类意图均命中预期类别
关键信息命中率	7/7 = 100.0%	限行、12123、施工、车距、快处等关键词均命中
评测运行方式	`python eval/run_eval.py`	不需要浏览器操作，可重复执行

在基础 12 条评测外，本文进一步加入 `eval/intent_compare_set.jsonl`，扩展到 18 条意图分类问题，覆盖“从 A 到 B 堵不堵”、“没有预测文件能否直接说准确率”、“上传图片后能否检测标志”等更接近真实问法的样例。脚本 `eval/run_intent_compare.py`

会先运行本地规则分类器；若本机 Ollama 中的 `travelqa` 模型可用，还会用同一 few-shot prompt 执行 LLM 分类对比。当前验收环境中 Ollama 服务未启动，因此 LLM 分类分支记录为 `not_run`，但 prompt 和评测集已写入 JSON，便于在 Dify LLM 分类节点中复现。规则路由结果见表18。

表 **18**　扩展意图分类评测

指标	结果	说明
扩展样本数	18 条	六类意图，每类含边界问法
规则路由准确率	18/18 = 100.0%	已修正 “堵不堵” 实时路线漏判
LLM few-shot 分类	当前未运行	Ollama 未启动，脚本保留 prompt 与接口
输出文件	结果 JSON	保存每条规则预测与 LLM 状态

针对 RAG 检索质量，本文增加 RAG 专项评测脚本和标注集。评测集包含 20 条人工标注查询，每条给出应命中的官方公告或法规标题片段；脚本把检索结果的来源标题与标注片段匹配，计算 Top-1、Top-3 和 Top-5 来源命中率。该评测关注 “是否召回正确来源”，不把大模型最终生成文本纳入评分，因此能单独衡量检索模块质量。结果见表19。

表 **19**　**RAG** 来源 **Top-K** 命中率

指标	结果	说明
评测样本数	20 条	覆盖 15 条交管公告、3 条法规和 2 条泛化查询
Top-1 命中率	20/20 = 100.0%	正确来源均排在第 1 位
Top-3 命中率	20/20 = 100.0%	正确来源均进入前 3
Top-5 命中率	20/20 = 100.0%	正确来源均进入前 5
输出文件	RAG 评测 JSON	保存每条问题的命中排名和 Top-5 标题

为说明 Hybrid Search 和 Reranker 的实际作用，脚本同时输出了小规模消融结果。表20 显示，在同一 20 条评测集上，当前方案 Top-1 命中 20/20；去掉 Reranker 后 Hybrid 检索 Top-1 降为 19/20；仅使用向量检索且不重排时 Top-1 降为 18/20。由于本评测集以

具体路名、区域名和公告标题为主，BM25 与 Reranker 对首位排序更有帮助。

表 **20**　**RAG** 检索消融结果

检索方案	Top-1	Top-3	Top-5
Hybrid + Reranker	20/20	20/20	20/20
Hybrid，无 Reranker	19/20	19/20	19/20
纯向量，无 Reranker	18/20	20/20	20/20

为了避免 20 条来源命中率评测过于友好，本文新增 RAG 压力集标注文件和配套评测脚本。压力集包含 8 条可回答问题和 4 条负例问题：可回答问题继续要求 Top-K 命中正确来源；负例问题包括广州、北京、成都不存在路名和强时效活动问题，要求检索结果不要出现精确错误匹配，且最高相似分低于阈值。结果见表21。

表 **21**　**RAG** 压力集与边界评测

指标	结果	说明
可回答正例 Top-1/3/5	8/8，8/8，8/8	官方公告和法规问题均命中正确来源
负例边界通过率	2/4 = 50.0%	广州、北京问题通过；成都近似时效问题仍会召回相近公告
总通过率	10/12 = 83.3%	比友好集更能暴露边界缺口
输出文件	压力集结果 JSON	保存每条 Top-5 标题和分数

需要说明的是，当前 RAG 评测仍以来源召回和边界检索为主，适合验证公告和法规是否被正确召回，以及知识库对外地或未覆盖问题是否容易误召回；若要进一步评估最终答案事实性，还需要人工标注标准答案，并加入回答准确率、引用正确率和有无 RAG 的生成质量对比。压力集暴露出的主要问题是：当问题仍含有 “成都” 和 “交通管制” 等强领域词，但具体事件并不在知识库中时，检索器会返回语义相近的成都公告。因此，后续应在回答层加入地域、事件名和发布时间守门规则，对最高分低或实体不一致的结果明确提示 “知识库未覆盖该问题”。

8.4 端到端延迟测试

在当前本地环境中，使用 `TravelQAApp.ask()` 对六类典型问题各测试一次，得到表22。该测试包含意图识别、检索或工具调用、模型组织回答与 panel 构造，不包含浏览器渲染时间。

表 **22**　典型问题端到端响应时间

问题类型	识别意图	响应时间 / s	panel 类型
尾号限行 RAG	traffic-info	2.934	rag
雨天出行建议	travel-advice	3.215	rag
天气问答	weather	2.374	无右侧 panel
路线规划	route	3.258	route
交通流预测	forecast	2.618	forecast
识别能力问答	detection	2.054	detection

从结果看，RAG 和路线规划略慢，主要原因是需要额外完成向量检索、BM25 重排或地图工具调用；预测和识别能力问答主要读取本地结果文件，因此耗时更低。若部署到真实并发场景，需要进一步使用 `locust`、`wrk` 或 `pytest-benchmark` 做并发压测，并把模型推理、RAG 检索和外部 API 调用分别计时。

8.5 界面与运行截图

图5 展示了从天府广场到春熙路的路径规划结果。中间问答区给出路线摘要和主要步骤，右侧面板调用高德地图显示起点、终点、路径轨迹、距离和预计耗时。



图 **5**　天府广场到春熙路路线规划界面

图6 展示了从西南交大犀浦校区到万象城的另一条路线规划。该截图说明系统不局限于示例问题，而是能够根据自然语言自动抽取起终点并生成路线证据面板。

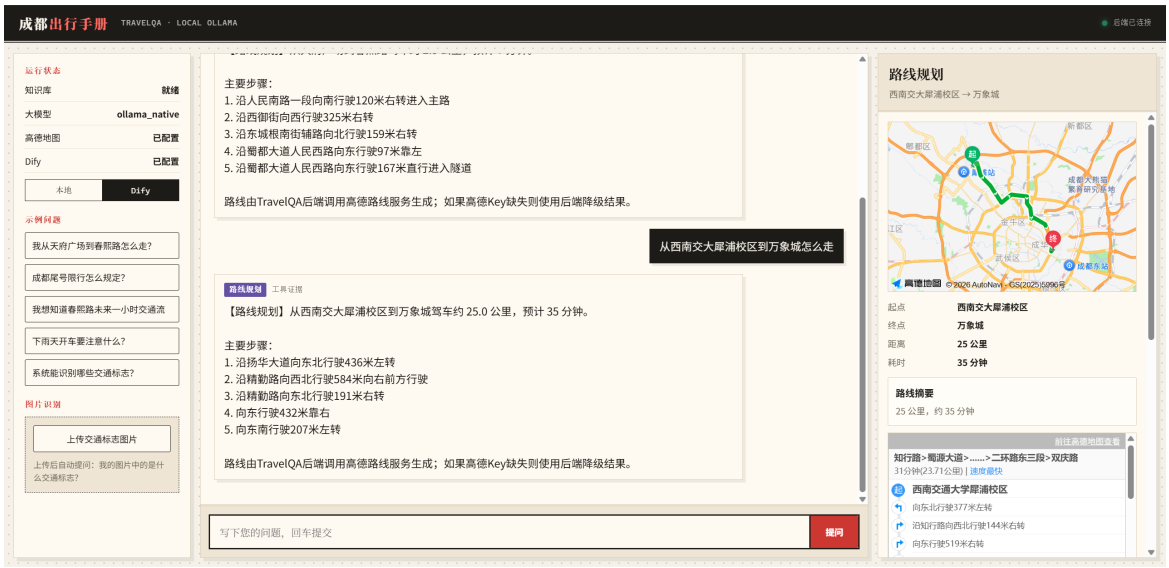


图 **6**　西南交大犀浦校区到万象城路线规划界面

图7 展示了尾号限行类强时效问题的回答。由于本地知识库未命中当前有效依据，系统将知乎搜索结果作为参考展示，并在回答中明确说明其仅供参考、不代表官方结论。



图 **7**　尾号限行问题的知乎补充参考界面

图8 展示了本地知识库命中的 RAG 问答。用户询问熊猫基地相关交通管理问题时，系统优先使用本地公告片段，并在右侧展示来源标题、发布时间、正文摘要和来源信息。

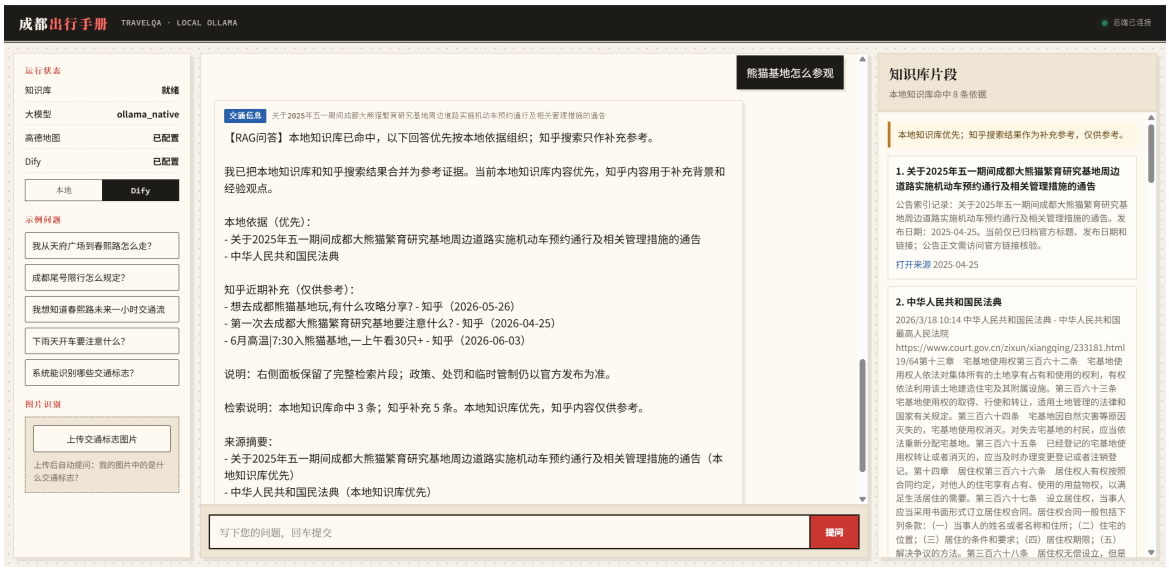


图 **8**　本地 **RAG** 命中与证据片段展示

图9 展示了交通流预测问答。系统将春熙路映射到 PEMS08 的演示节点，调用 STAE-former 预测未来 60 分钟交通流，并在右侧绘制趋势曲线和各时间步预测值。

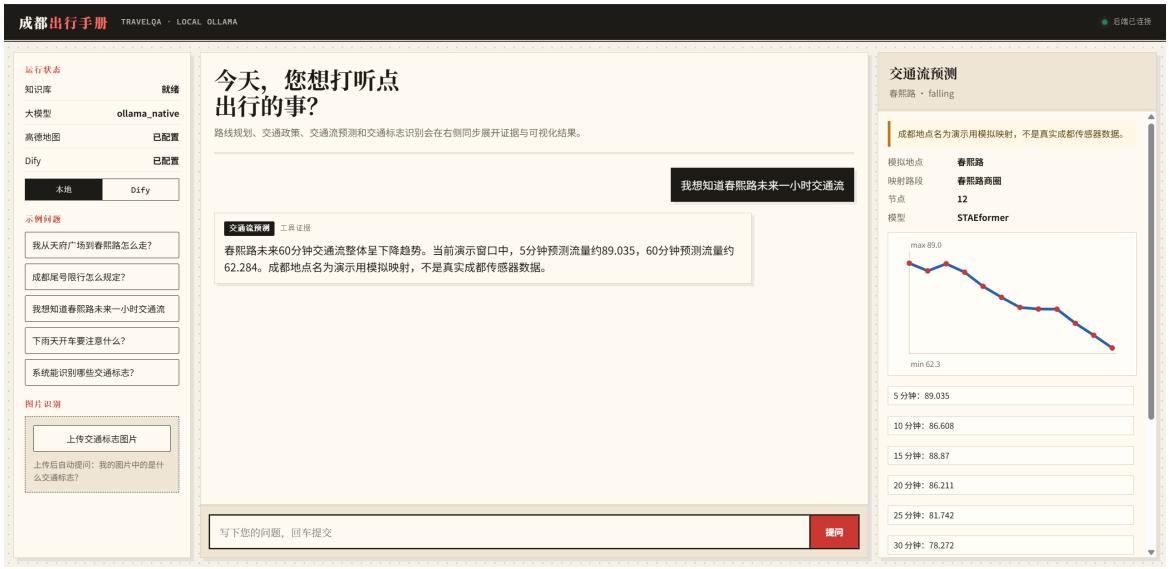


图 **9**　春熙路未来一小时交通流预测界面

图10 展示了交通标志图片识别功能。用户上传 “禁止驶入” 标志图片后，系统在中间问答区给出识别结论，右侧面板展示标注图、类别、置信度和边界框坐标。该界面体现了本项目 “用户问答 + 工具证据 + 可视化面板” 的三栏设计思想。



图 **10**　交通标志识别问答界面

图11 展示了 Dify 工作流的运行日志。日志中可以看到路线规划、交通流预测、限行问答等多类请求均由 `travelqa-web` 用户触发，并获得成功状态，说明前端、后端代理和 Dify 工作流之间的调用链路已经打通。

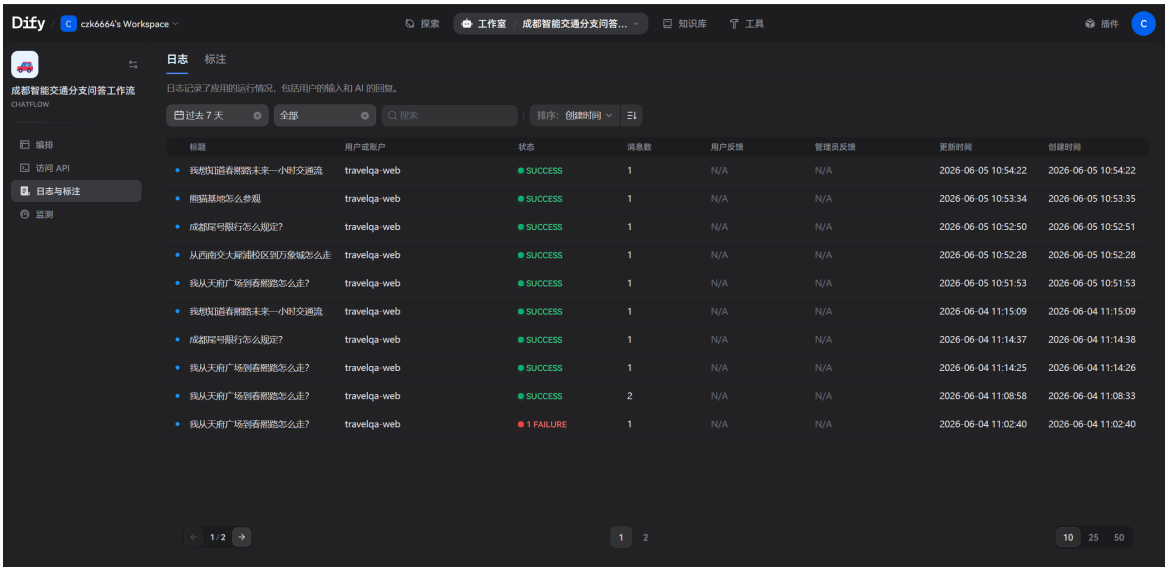


图 **11**　**Dify** 工作流日志页面

图12 展示了 Dify 监控页面。监控面板记录会话数量、活跃用户、平均互动数和 Token 输出速度等指标，可用于观察系统在演示和测试过程中的运行状态。

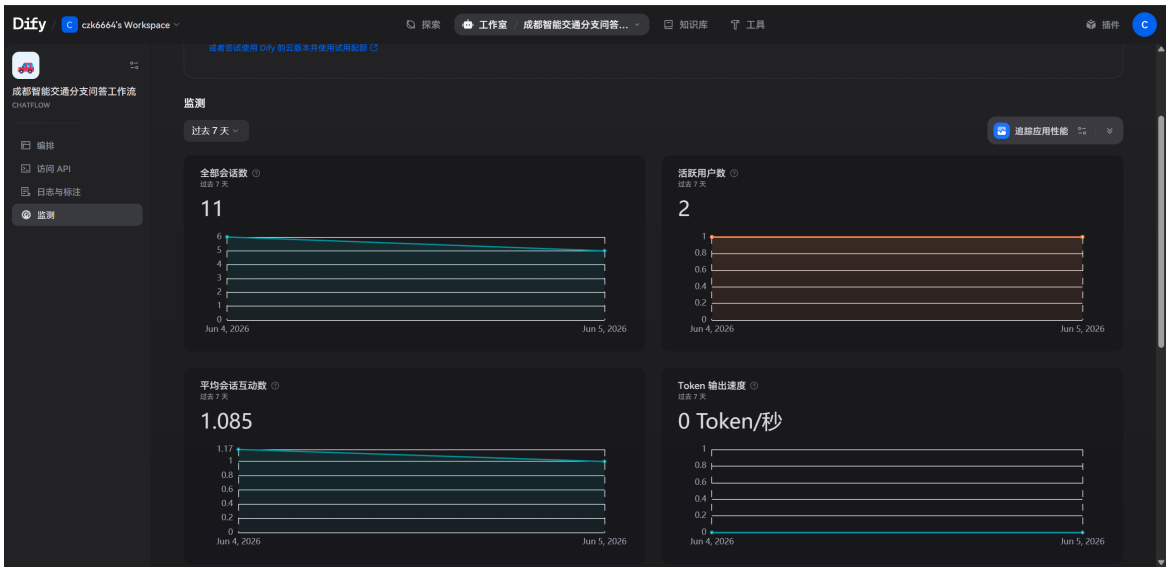


图 **12**　**Dify** 工作流监控页面

8.6 错误处理验证

系统对常见异常进行了处理：

- 高德 JS Key 缺失：前端地图降级为路线摘要和 polyline 简图。
- 高德 Web 服务失败：后端返回 `available=false` 和错误原因。
- YOLO 模型缺失：返回模型不可用提示。
- Dify 未发布：后端返回 Dify 调用失败，提示 `Workflow not published`。
- Docker 未启动：一键启动脚本尝试启动 Docker Desktop 并等待就绪。

九、 结果分析

从作业要求覆盖角度看，系统已经完成 Dify 工作流编排、DeepSeek 1–7B 级基座模型微调、本地 RAG 出行知识库、交管信息问答、天气问答、出行建议、路径规划、交通流预测问答和交通目标识别理解问答等核心内容。与普通聊天问答相比，本系统最大的特点是每轮回答都可以附带结构化证据或可视化面板，用户能够看到模型回答背后的工具结果。

从工程实现角度看，系统采用 FastAPI 统一封装所有工具，使本地 Web 前端和 Dify 工作流可以复用同一套后端能力。Dify 模式并未破坏本地前端体验，而是通过后端代理和本地 panel 重建，实现了工作流生成回答与本地可视化展示的结合。

从智能交通应用角度看，系统将交通政策知识、地图路线服务、交通流预测和交通标志识别放入同一交互界面中，体现了智能交通系统中信息融合和多源协同的思想。当前交通流预测使用成都地点到 PEMS08 节点的演示映射，因此它证明的是“大作业 1 预

测模型作为工具节点被问答系统调用并可视化”，而不是证明系统已经具备真实成都道路的交通流预测能力。报告和前端均明确标注该限制，后续只需替换为成都真实检测器或交通指数数据，即可复用现有 `ForecastTool`、Dify 节点和右侧折线图 panel。

从评估结果看，当前系统已经具备基础可验证性：RAG 知识库规模、LoRA 数据规模、离线功能评测、扩展意图评测、RAG Top-K 来源命中率、RAG 消融结果、RAG 压力集、端到端延迟、A/B 规则化评分和 20 条人工盲评均可在项目目录中复查。相比只展示截图，这些结果更能说明系统链路真实存在。但也应看到，当前评估仍以课程原型为边界：RAG Top-K 评测验证的是来源召回，不等价于最终回答全自动事实核验；A/B 评分验证的是工具调用意识、拒答边界和思维泄漏控制，人工盲评样本也只是小规模复核。

十、 不足与改进方向

当前系统仍存在以下不足：

1. 交通流预测尚未接入真实成都传感器数据，当前为成都地点名到 PEMS08 节点的演示映射，因此不能把预测值解释为真实成都路段流量。
2. `DeepSeek-R1-Distill-Qwen-7B` 模型卡参数量为 7.62B，属于 7B 级蒸馏模型；若评分严格限定 1–7B 字面范围，应准备 1.5B 或 `deepseek-llm-7b-chat` 替代方案。
3. RAG 已完成 20 条来源 Top-K、12 条压力集和小规模消融，但压力集显示 4 条负例中仍有 2 条近似成都时效问题会召回相近公告，说明还需要实体一致性和时效性守门。
4. 微调已完成 100 条 A/B 规则化评分和 20 条人工盲评复核，但人工评分样本仍偏小，后续仍可按流畅度、完整性和引用表达质量扩大双盲评测。
5. 本地知识库需要定期更新，否则政策类问题仍可能存在时效性不足。
6. 知乎搜索结果质量不稳定，需要进一步加入时间、来源可信度和内容相关性过滤。
7. 图片识别依赖本地 YOLO 权重和运行环境，首次部署成本较高。
8. Dify 工作流目前主要调用本地工具接口，尚未充分利用 Dify 原生知识库能力。
9. 前端地图展示依赖高德 JS API Key，未配置时只能使用降级展示。

后续可以从以下方向继续完善：

- 接入成都市真实道路传感器或开放交通指数数据。
- 建立定时爬取和自动重建知识库任务。
- 在 Dify LLM 分类节点中复用 `run_intent_compare.py` 的 few-shot prompt，补充规则分类与 LLM 分类的在线对比结果，并保留规则路由作为离线降级路径。
- 扩展 RAG 压力集并增加实体一致性校验、发布时间校验和低分拒答阈值，补充最终回答准确率、引用正确率和人工事实核验。

- 在现有 `score_ab_eval.py` 与 `ab_blind_review_20.json` 基础上扩大人工评分样本，继续报告微调前后在流畅度、完整性和引用表达质量上的差异。
- 在 Dify 中进一步配置原生知识库、模型供应商和工具插件。
- 增加用户会话记忆和多轮路线澄清能力。
- 将 YOLO 图片识别扩展为视频流实时检测。

十一、 总结

本文按照“大小模型协同的智能交通出行问答系统”课程设计要求，完成了一个面向成都市出行场景的问答原型。系统以 Dify 工作流和 FastAPI 后端为核心，采用 DeepSeek 开源 1–7B 级模型 `DeepSeek-R1-Distill-Qwen-7B` 作为基座并完成 LoRA/QLoRA 微调，网络采集成都市交管局信息发布栏并结合交通法规资料构建本地 RAG 出行知识库，同时整合高德地图 API、天气工具、知乎搜索补充、STAEformer 交通流预测和 YOLO 交通目标识别。前端通过三栏式工作台实现问答、证据和可视化结果同步展示，用户不仅可以获得自然语言回答，还可以在右侧查看地图路线、知识片段、预测曲线和图片标注图。

在 Dify 集成方面，本文构建了中文节点的分支式工作流，使系统具备可视化编排能力。通过后端代理设计，系统保护了 Dify App Key，并保证 Dify 模式下右侧面板仍保持本地模式的展示形式。项目还提供了一键启动和停止脚本，降低演示和部署复杂度。

总体而言，本项目实现了从交管信息采集、本地 RAG 检索、基座模型微调、外部 API 调用、小模型预测识别到大模型回答生成的完整闭环，覆盖题目要求的基础问答功能和前两个大作业模型成果集成，能够较好体现智能交通课程设计中对多源数据融合、大小模型协同和工程系统实现的综合要求。

参考文献

[1] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020.

[2] DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948, 2025.

[3] ModelScope. DeepSeek-R1-Distill-Qwen-7B 模型介绍. `https://modelscope.cn/models/deepseek-ai/DeepSeek-R1-Distill-Qwen-7B`

[4] Robertson S, Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 2009.

[5] Dify Documentation. Dify Workflow and Chatflow Documentation. https://docs.dify.ai/

[6] 高德开放平台. 高德 Web 服务 API 与 JavaScript API 文档. https://lbs.amap.com/

[7] Liu H, et al. Spatio-Temporal Adaptive Embedding Makes Vanilla Transformer SOTA for Traffic Forecasting. CIKM, 2023.

[8] Jocher G, Chaurasia A, Qiu J. Ultralytics YOLO. https://github.com/ultralytics/ultralytics

[9] Zhu Z, Liang D, Zhang S, et al. Traffic-Sign Detection and Classification in the Wild. CVPR, 2016.

[10] 知乎开放平台. 知乎搜索 API 文档.